

北京交通大学软件综合实训

20_北京交通大学就餐仿真系统设计规格说明书

Design Specification

项目名称

北京交通大学就餐仿真系统

分组编号

20

文档版本

V1.0

编制说明

依据课程简介、交付物要求明细、文档命名规范和文档格式基础要求编制。

北京交通大学计算机科学与技术学院

2026 年 03 月 17 日

目 录

1 系统总体目标	1
1.1 项目背景	1
1.2 总体建设目标	1
1.3 建设范围与边界	1
2 系统开发架构	3
2.1 架构选择	3
2.2 部署拓扑	3
2.3 架构优势	4
3 系统开发环境	5
3.1 开发环境选型	5
3.2 开发语言与依赖	5
3.3 协作与测试工具	6
3.4 LLM 接入策略	6
4 系统架构	7
4.1 模块划分	7
4.2 协同流程	8
4.3 模块接口与数据交换	9
4.4 接口设计	9
5 软件原型	11
5.1 原型设计目标	11
5.2 核心界面原型	11
5.3 界面流程	11
6 数据模型	14
6.1 数据分类与来源	14
6.2 核心数据对象	14
6.3 数据流转过程	16
6.4 持久化设计	16

说明：目录页码按正文起始页重新编号，封面与目录不计入正文页码。

1 系统总体目标

北京交通大学就餐仿真系统面向学校食堂中午与晚间高峰时段的排队和就座问题，围绕“学生到来—窗口排队—取餐—入座—就餐—离场”的完整过程，构建可配置、可运行、可分析、可解释的仿真软件。系统设计遵循课程“从较小规模系统出发、在设计中保留扩展空间、逐步迭代优化”的要求，优先完成稳定的闭环仿真，再在此基础上增加优化推荐与 LLM 解释能力。

1.1 项目背景

食堂就餐效率受窗口数量、窗口服务效率、学生到达峰值、座位数量与周转速度等多种因素共同影响。仅凭主观观察难以判断“究竟应该加窗口、扩座位，还是采用错峰策略”，因此需要建立一个能反复实验、量化比较和可视化展示的就餐仿真系统。课程材料要求同学在立项阶段回答系统长什么样、用户如何一步步使用、模块如何划分、模块之间如何协同以及数据如何产生和流转，本说明书正是对这些问题的系统化回答。

1.2 总体建设目标

- 目标一：建立分钟级离散时间仿真模型，能够根据窗口数、座位数、到达率、平均打饭时长、平均就餐时长等参数，模拟典型就餐过程。
- 目标二：提供图形化交互界面，使用户能够完成参数输入、仿真启动、暂停、单步调试、运行观察、结果导出与多方案对比。
- 目标三：自动输出关键指标，包括平均等待时间、峰值排队长度、累计接待人数、空座位数变化、窗口利用率等，为后续报告编写与答辩演示提供直接素材。
- 目标四：在核心仿真稳定后，增加优化推荐模块，自动计算较优的窗口、座位与错峰配置，并结合 LLM 对调整原因和策略进行自然语言解释。

1.3 建设范围与边界

本项目聚焦单食堂、单餐段、多窗口、多座位场景，先实现可运行的局部仿真系统，不直接覆盖支付系统、后厨生产排班、校园地图导航等复杂业务。首版采用分钟级时间步推进，不追求完全贴近真实世界的细粒度秒级行为，而是通过统一的数据口

径获得可解释、可对比、可复现实验结果。LLM 解释模块属于增强模块；当外部模型不可用时，系统仍然输出结构化推荐与规则化说明，保证课程验收所需的核心能力不受影响。

设计原则：系统以“可实现、可验证、可展示、可迭代”为首要目标，先完成配置、仿真、记录、分析与展示五环闭环，再扩展优化与解释能力。

2 系统开发架构

2.1 架构选择

本项目选择 B/S（Browser / Server）架构，并采用“浏览器前端 + 本地后端仿真服务”的部署方式。用户通过浏览器访问界面完成配置和观察；后端负责仿真计算、记录落盘、指标分析、优化推荐和 LLM 解释。相较于传统 C/S 架构，B/S 架构在课堂展示、跨平台运行、组内协作和模块边界划分方面更有优势，也更适合课程强调的界面展示与全流程文档化。

表 2-1 架构选型比较

评估维度	B/S 架构表现	C/S 架构表现	本项目结论
课堂演示	浏览器直接访问，展示友好	需安装客户端，不利于演示切换	选 B/S
跨平台部署	Windows / macOS / Linux 均可使用浏览器访问	不同平台需分别适配客户端	选 B/S
前后端分工	界面与仿真逻辑边界清晰，便于 2~3 人并行开发	界面与逻辑可能耦合较紧	选 B/S
后续扩展	便于接入 REST API、导出接口与可选 LLM 服务	扩展通常需要修改客户端	选 B/S

2.2 部署拓扑

系统部署采用本地单机方式：浏览器负责参数输入与结果展示；后端服务运行在本地 Python 环境中，统一暴露 REST 接口；过程记录和分析结果存储在 SQLite 或导出的 CSV 文件中；若启用 LLM 解释模块，则由后端通过适配器调用兼容接口。此方案既便于课程演示，也便于在后续阶段编写部署说明和使用手册。

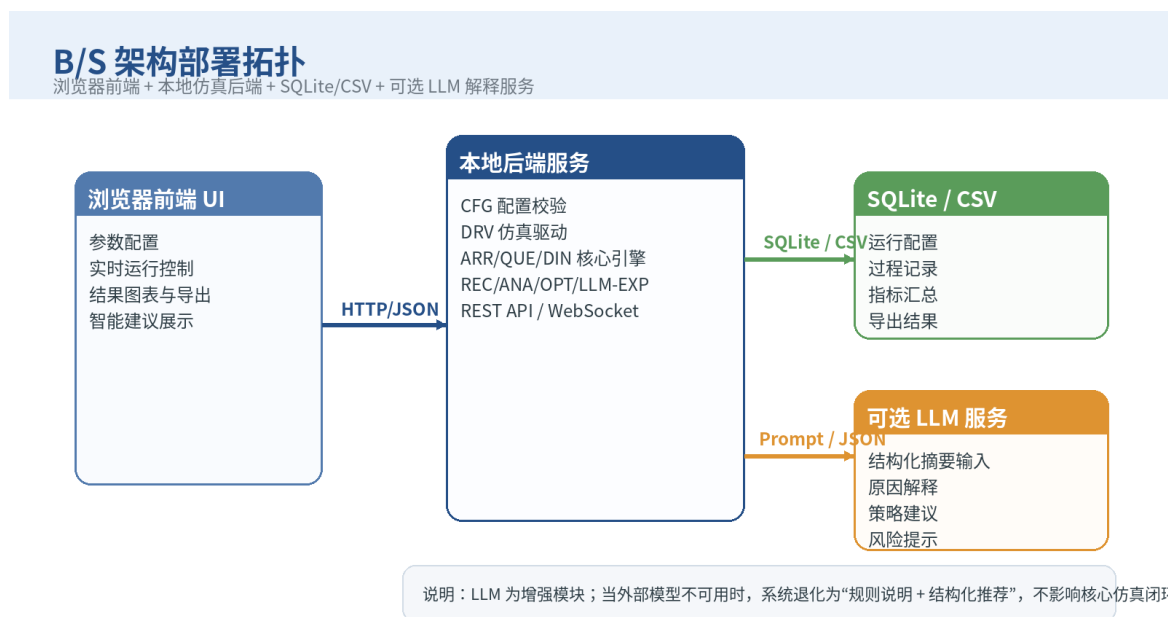


图 2-1 系统 B/S 架构部署拓扑

2.3 架构优势

- 界面层与仿真层分离，便于小组成员按“前端可视化 / 后端仿真 / 数据分析与文档”进行分工。
- 前端页面天然适合展示队列长度、座位矩阵、趋势曲线、推荐结果和解释文本，契合课程答辩场景。
- 后端统一掌管数据契约和仿真时钟，利于后续进行单元测试、联调测试和批量实验。
- LLM 接入点固定在后端解释层，不会反向污染核心仿真模块，便于控制复杂度和风险。

3 系统开发环境

3.1 开发环境选型

根据课程对语言和架构“不做硬性要求”的特点，本项目选择成熟、轻量且便于快速迭代的技术栈。开发环境兼顾“课程能按周推进、交付物可复现、成员上手门槛适中”三个方面。

表 3-1 开发环境配置

类别	选型方案	说明
操作系统	Windows 11 / Ubuntu 22.04	组员可按个人电脑环境选用，保证浏览器和 Python 环境可用。
开发工具	PyCharm 或 VS Code	便于进行 Python 调试、前端页面开发与 Git 协作。
原型与绘图工具	Figma、draw.io 或 ProcessOn	用于软件原型、模块图、数据流图绘制。
版本管理	Git + Gitee/GitHub	用于代码协同、分支管理与提交记录留痕。
后端运行环境	Python 3.11 + venv	统一后端开发环境，降低依赖冲突风险。
前端运行环境	Node.js（可选）	若采用 Vue 3 方案，则用于构建前端页面；也可先用静态页面/模板降低复杂度。

3.2 开发语言与依赖

后端采用 Python 语言实现核心仿真与数据分析模块，前端采用 HTML5 / CSS3 / JavaScript（或 Vue 3）实现交互页面。之所以优先选择 Python，是因为其在离散事件模拟、数据处理、脚本化测试、CSV/SQLite 操作方面具有较高开发效率，能够更快完成课程所要求的闭环系统。

表 3-2 开发语言与依赖组件

层次	主要技术/依赖	用途
前端层	Vue 3、Element Plus、ECharts（可按复杂度裁剪）	实现参数配置、实时运行页、指标图表和推荐解释页。

服务层	FastAPI、Uvicorn、Pydantic	提供 REST 接口、参数校验、DTO 序列化与调试接口。
仿真与分析层	Python 标准库、pandas（可选）	完成仿真逻辑、过程统计、结果聚合与导出。
存储层	sqlite3、CSV、JSON	存储配置快照、过程记录、指标汇总与推荐结果。
测试层	pytest	进行模块级单元测试与集成阶段回归验证。

3.3 协作与测试工具

课程过程考核强调沟通记录、阶段任务推进和测试证据，因此项目除代码环境外，还需要配套协作与验证工具。组内统一使用 Git 记录代码提交，使用共享云盘或仓库存放图表、原型与文档素材，使用 issue 或周计划表跟踪阶段任务。对于测试，后端核心模块尽量采用可编程的单元测试；界面层则采用“人工操作脚本 + 截图留存”的方式辅助验证。

- 单元测试重点覆盖 ARR、QUE、DIN、REC、ANA 等逻辑性强、输入输出清晰的模块。
- 联调测试以“配置—运行—导出—分析—推荐”完整链路作为主场景。
- 所有阶段文档统一从源文件导出为 PDF，并按课程命名规范进行提交。

3.4 LLM 接入策略

LLM 不直接参与核心仿真计算，而是在分析与优化结果生成后，读取结构化摘要并输出自然语言解释。此设计一方面可以保证“算得对”和“讲得明白”分层实现，另一方面也能将外部模型服务对系统稳定性的影响控制在可接受范围内。工程上为 LLM 预留统一适配接口，当模型不可用时，解释层返回模板化说明。

- 输入：基准方案、推荐方案、关键指标变化、瓶颈分析结论、Top-K 备选方案。
- 输出：根因解释、推荐理由、备选策略、风险提示、适用条件。
- 容错：LLM 超时或失败时，使用规则模板生成说明，不阻塞核心功能。

4 系统架构

4.1 模块划分

依据课程对模块划分、协同方式、数据流转和接口形式的要求，系统按职责被划分为 10 个模块：CFG 初始配置模块、DRV 仿真驱动模块、ARR 人员到来模块、QUE 窗口排队与打饭模块、DIN 就餐与离场模块、REC 过程记录模块、ANA 数据分析模块、OPT 优化推荐模块、LLM-EXP 解释模块、UI/VIS 可视化交互模块。各模块围绕统一的数据契约运行，既能形成稳定主链路，也便于按模块完成开发、单测与联调。

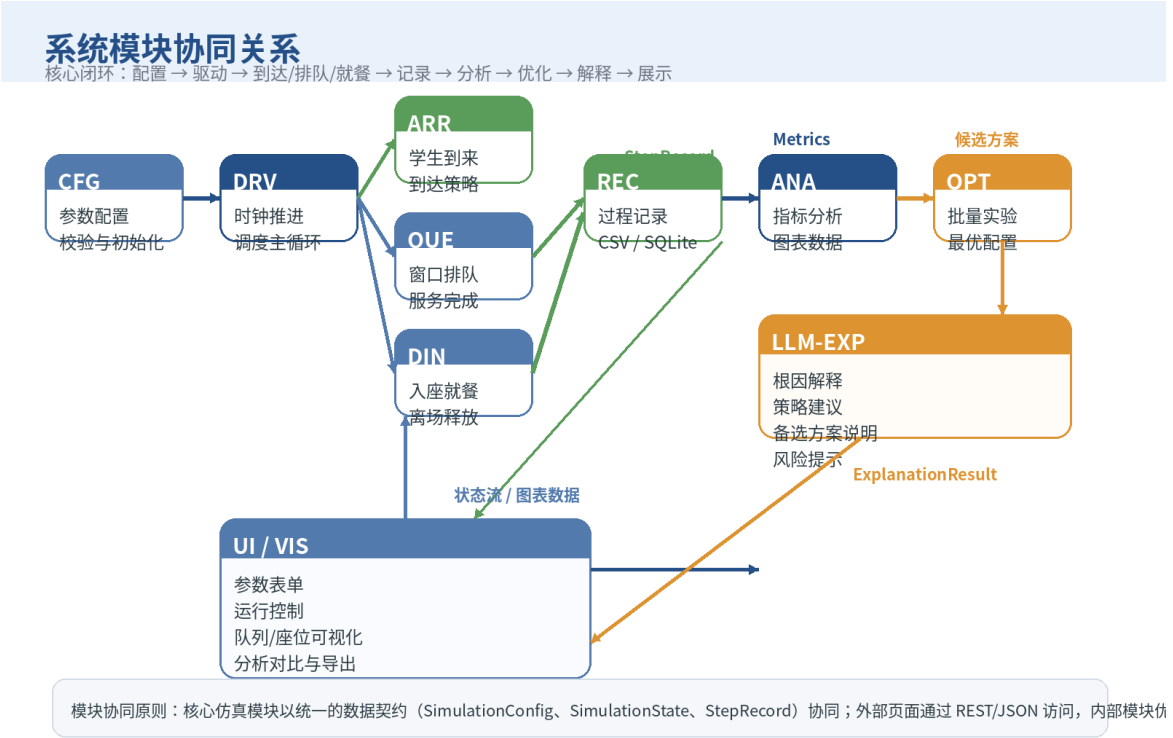


图 4-1 系统模块协同关系图

表 4-1 系统模块划分与职责

模块	核心职责	主要输入	主要输出
CFG	接收并校验参数，生成可复现实验配置	用户表单、默认配置、历史配置	SimulationConfig、ValidationResult
DRV	维护仿真时钟，按步骤调度各核心模块	SimulationConfig、控制命令	SimulationState、StepRecord 流
ARR	根据到达率/到达分布生成到来学生	当前时刻 t、到达策略、随机种子	新到学生列表

QUE	管理窗口队列、服务完成与排队统计	学生列表、窗口状态、服务参数	完成取餐学生、队列长度、窗口服务统计
DIN	实现入座、就餐、离场与座位释放	完成取餐学生、座位状态、就餐参数	空座位数、离场人数、座位快照
REC	记录每一步状态与关键事件	StepRecord	CSV/SQLite 记录、日志
ANA	聚合过程记录并计算指标	过程记录、配置快照	MetricsSummary、图表数据
OPT	批量实验、评分排序并选择推荐方案	候选配置集合、指标结果	OptimizationResult
LLM-EXP	生成推荐原因与调整策略说明	ExplanationRequest	ExplanationResult
UI/VIS	展示参数、状态、图表和推荐信息	后端 DTO、导出文件	交互界面、导出操作

4.2 协同流程

系统协同分为四个阶段。第一阶段是仿真初始化：用户在配置页提交参数，CFG 完成合法性检查，DRV 根据配置创建初始状态。第二阶段是单步推进：每个时间步由 DRV 依次调用 ARR 生成新到学生，调用 QUE 完成分配与打饭服务，再由 DIN 更新座位占用和离场结果，同时 REC 记录新的状态快照。第三阶段是分析优化：仿真结束后，ANA 汇总过程记录计算指标；若用户启用智能推荐，OPT 基于多组候选配置重复调用仿真链路，给出评分排序与推荐配置。第四阶段是解释展示：LLM-EXP 读取结构化摘要生成解释文本，UI 将过程图表、推荐结果和解释内容一并展示给用户。

- 主链路数据顺序为：Configuration → State → StepRecord → Metrics → Optimization → Explanation。
- 主链路控制顺序为：提交参数 → 启动/暂停/单步 → 仿真结束 → 分析 → 推荐 → 导出。
- UI 与后端通过 REST/JSON 协同；后端内部模块以函数/服务调用和数据对象协同。

4.3 模块接口与数据交换

为降低集成阶段返工风险，系统在立项阶段就冻结核心数据契约。对外采用 JSON DTO，对内采用 Python 数据对象或服务方法参数。模块之间的主要数据交换关系如下表所示。

表 4-2 模块接口与数据交换

调用方	被调用方	交换数据	接口形式
UI	CFG	配置表单 JSON	REST: POST /api/config/validate
DRV	ARR	当前时刻、到达策略、随机种子	函数调用： generate_arrivals(t, cfg)
DRV	QUE	新到学生列表、窗口状态、服务参数	函数调用： service_step(state, arrivals, cfg)
DRV	DIN	完成取餐学生、座位状态、就餐参数	函数调用： seat_and_dine(state, served, cfg)
DRV	REC	StepRecord	服务调用： write_step(record)
ANA	REC	过程记录集 / 运行编号	DAO 查询或 CSV 读取
OPT	DRV/ANA	候选配置集合、评分函数	批量服务调用
LLM-EXP	OPT/ANA	ExplanationRequest	服务调用或 HTTP 适配器
UI	ANA/OPT/LLM-EXP	指标、推荐、解释 DTO	REST: GET/POST JSON

4.4 对外服务接口

为了支持浏览器界面、联调测试与后续部署，本项目在后端统一提供以下核心接口。接口名称可在开发阶段根据实际实现调整，但数据语义与请求/响应边界在立项阶段即固定。

表 4-3 对外服务接口设计

接口	方法	说明
----	----	----

/api/config/validate	POST	校验参数配置并返回错误/警告信息。
/api/sim/run	POST	启动一轮完整仿真，返回运行编号和初始化状态。
/api/sim/step	POST	执行单步仿真，便于调试与课堂演示。
/api/run/{id}/records	GET	获取过程记录或导出 CSV。
/api/run/{id}/metrics	GET	获取分析指标与图表数据。
/api/optimize/recommend	POST	对候选参数执行批量实验并返回推荐方案。
/api/explain	POST	基于结构化摘要生成自然语言解释。
/api/export/{id}	GET	导出图表 PNG、摘要 JSON 或结果 CSV。

5 软件原型

5.1 原型设计目标

软件原型的目标不是一次性做成最终高保真界面，而是提前回答“系统长什么样、用户能做什么、页面如何跳转”这三个课程核心问题。因此，本项目原型遵循“操作路径短、关键状态可见、演示信息集中”的原则，优先保证参数配置页、实时运行页、分析对比页和推荐解释页四类页面的完整性。

- 参数输入要集中在一个页面内完成，避免过多页面跳转。
- 运行过程要同时呈现队列、座位和关键指标，便于理解系统行为。
- 分析和推荐页面既要给结论，也要说明推荐依据和备选策略。

5.2 核心界面原型

系统首版建议包含四个核心页面：参数配置页、实时运行页、结果分析页和优化推荐页。参数配置页负责实验输入；实时运行页负责展示队列与座位联动过程；结果分析页负责显示指标和多次实验比较；推荐解释页负责展示自动推荐结果以及 LLM 解释文本。以下原型图展示了页面布局重点。



图 5-1 参数配置页原型



图 5-2 实时运行页原型



图 5-3 优化推荐与解释页原型

表 5-1 核心页面功能与操作

页面	用户可执行操作	主要结果/跳转
参数配置页	输入窗口数、座位数、到达率、打饭时长、就餐时长、仿真时长、随机种	通过“开始仿真”进入实时运行页；校验失败则停留当前页并提示问题。

	子和优化候选范围；执行参数校验；加载默认场景。	
实时运行页	启动、暂停、继续、单步、重置；查看队列柱状图、座位网格、关键指标卡片；导出过程记录。	运行结束自动跳转到结果分析页，或返回参数配置页重设参数。
结果分析页	查看指标汇总、趋势曲线、基准与对比实验结果；选择是否发起推荐计算。	点击“生成推荐”进入优化推荐页；点击“重新实验”回到参数配置页。
优化推荐页	查看推荐窗口/座位/错峰配置、LLM 原因解释、备选策略和风险提示；导出摘要。	可返回结果分析页，也可返回参数配置页重新构造方案。

5.3 页面跳转关系

页面跳转采用简单清晰的单主线结构：用户总是从参数配置页进入系统，提交参数后进入实时运行页；仿真完成后进入结果分析页；若用户需要更进一步的决策支持，再由结果分析页进入优化推荐页。任何页面都允许返回参数配置页重新调整方案，从而支持课堂上的多轮实验演示。

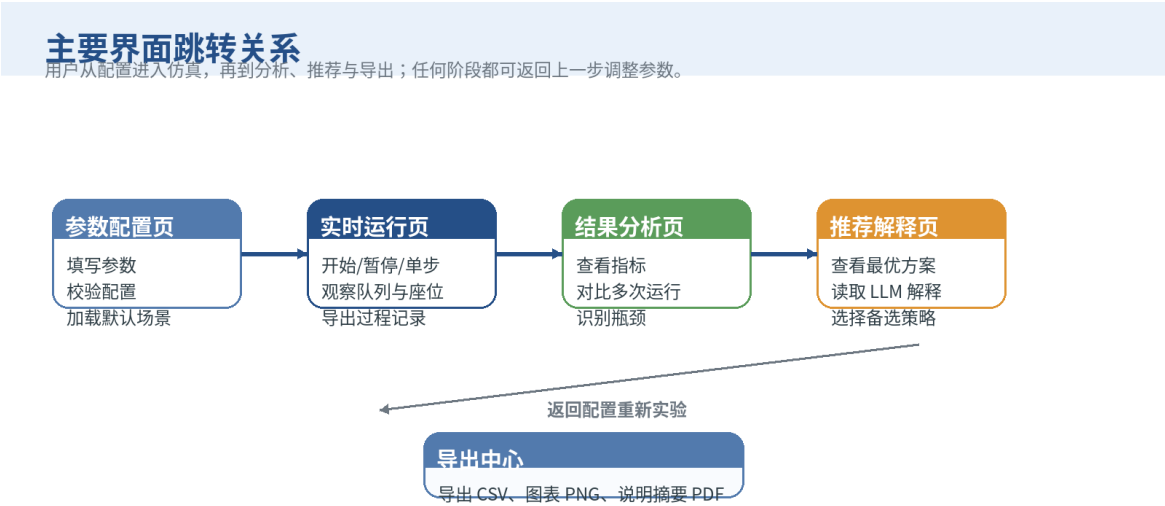


图 5-4 页面跳转关系图

6 数据模型

6.1 数据分类与来源

系统数据分为配置数据、运行态数据、过程记录数据、分析结果数据、优化结果数据、解释结果数据和导出数据七类。其中，配置数据由用户在界面输入形成；运行态数据由仿真引擎在内存中持续维护；过程记录由 REC 模块逐步落盘；分析结果、推荐结果和解释结果由 ANA、OPT 与 LLM-EXP 模块在仿真结束后生成；导出数据由 UI 和后端接口在用户触发导出时生成。

表 6-1 数据分类与来源

数据类别	产生模块/位置	形成时机	流转去向	存储方式
配置数据	UI + CFG	用户提交参数时	DRV、OPT、REC	JSON / SQLite
运行态数据	DRV、ARR、QUE、DIN	每个时间步推进时	REC、UI	内存对象
过程记录	REC	每一步仿真完成后	ANA、导出中心	SQLite / CSV
分析结果	ANA	仿真结束或用户手动分析时	UI、OPT、LLM-EXP	SQLite / JSON
优化结果	OPT	批量实验完成后	UI、LLM-EXP	SQLite / JSON
解释结果	LLM-EXP	收到 ExplanationRequest 后	UI、导出中心	SQLite / JSON
导出文件	UI + Export 接口	用户点击导出时	课程文档与演示材料	CSV / PNG / PDF

6.2 核心数据对象

为了保证模块之间的数据口径统一，系统围绕以下核心对象展开设计：SimulationConfig 用于描述单次仿真的输入参数；SimulationState 用于表示当前时刻的整体状态；StepRecord 用于记录每一步的快照；MetricsSummary 用于保存分析结果；

OptimizationResult 用于保存批量实验的评分排序结果；
ExplanationRequest/ExplanationResult 用于衔接结构化结果和自然语言解释。

表 6-2 核心数据对象说明

对象	关键字段	说明
SimulationConfig	num_windows、num_seats、arrival_schedule、service_time_mean、dining_time_mean、duration_min、seed	描述单次仿真的核心输入。
SimulationState	t、queues、window_busy_state、seat_matrix、waiting_for_seat_count、total_arrived、total_served	描述当前时刻的全局运行状态。
Student	student_id、arrival_time、queue_enter_time、service_start_time、service_end_time、seat_time、leave_time	用于精细追踪个体等待与逗留过程，可按需要裁剪字段。
StepRecord	run_id、t、arrived_count、queue_lengths、served_count、left_count、empty_seats、snapshot_json	记录每分钟状态和关键事件，是分析的原始依据。
MetricsSummary	avg_wait、peak_queue、throughput、seat_utilization、window_utilization、bottleneck_type	保存一次仿真的汇总指标。
OptimizationResult	best_config、best_score、top_k_configs、constraint_status	保存候选方案评分、排序与最优推荐。
ExplanationRequest/Result	baseline_config、best_config、metrics_delta、root_cause_summary、recommended_strategy、risk_notes	连接分析/优化层与 LLM 解释层。

6.3 数据流转过程

数据流转从参数配置开始：用户在前端配置表单中输入参数后，由 CFG 模块生成配置对象；DRV 读取配置并初始化状态；随后每个时间步都会产生新的状态快照，并由 REC 模块落盘；仿真结束后，ANA 读取过程记录生成指标；OPT 在需要时多次复用前述链路，对多组参数进行评分；最后，LLM-EXP 使用结构化摘要生成解释文本。数据流转既支撑系统运行，也直接为阶段报告、联调测试报告和总结材料提供证据。

数据生命周期与存储模型

配置数据、运行状态、过程记录、分析结果、优化结果与解释结果构成可追溯的数据闭环。

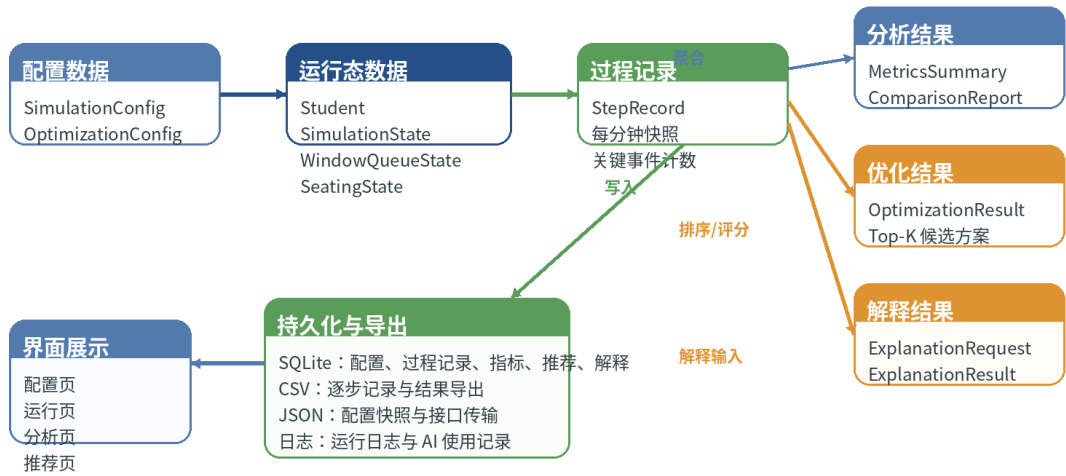


图 6-1 数据生命周期与存储模型

6.4 持久化设计

系统持久化采用“内存运行态 + SQLite 主存储 + CSV/JSON 导出”的组合方式。运行中的临时状态全部放在内存中，以保证仿真效率；结束后的结构化结果落入 SQLite，便于查询与多次实验对比；课程报告需要的过程记录和图表数据则导出为 CSV、JSON 或 PNG 文件。该设计既满足课堂演示和快速迭代，又能为集成阶段的联调测试和部署阶段的使用手册提供稳定的数据支撑。

表 6-3 持久化设计

存储对象	主要字段/内容	用途
run_config	run_id、created_at、config_json	保存单次运行配置快照，保证实验可复现。
step_record	run_id、t、arrived_count、queue_lengths_json、served_count_json、left_count、empty_seats、snapshot_json	保存逐步快照，为分析与回放提供数据。
metrics_summary	run_id、avg_wait、peak_queue、throughput、seat_utilization、window_utilization、bottleneck_type	保存汇总指标，便于结果比较。
optimization_result	opt_id、base_run_id、candidate_json、best_config_json、ranking_json	保存批量实验与推荐结果。

explanation_result	exp_id、run_id、request_json、 response_text、risk_notes、created_at	保存解释文本，支持再次查看和导出。
export files	CSV、JSON、PNG、PDF 摘要	用于课程文档、答辩演示和阶段归档。

数据口径要求：所有模块共享统一字段命名和时间单位，推荐以“分钟”为时间基准；任何统计指标若采用近似算法，需在分析报告中注明假设。